CLASE 3 - Sistemas Digitales

Semana 3: Arquitectura y programación de microcontroladores

1. Buses: lectura y escritura

Los buses permiten intercambiar información entre la CPU, la memoria y los periféricos.

- **Leer (READ):** obtener información almacenada en una posición de memoria o registro.
- **Escribir (WRITE):** guardar información en una posición de memoria o registro.

Para realizar estas operaciones se utilizan los buses de direcciones, datos y control.

2. Arquitecturas de computadores

Existen dos arquitecturas principales que se diferencian por la organización de la memoria y los buses.

Von Neumann

Utiliza una misma memoria para almacenar las instrucciones del programa y los datos.

- Un único espacio de memoria para programa y datos.
- Comparte el bus para acceder a instrucciones y datos.
- Puede ser más lenta porque las instrucciones y los datos compiten por el mismo camino de comunicación (cuello de botella de Von Neumann).

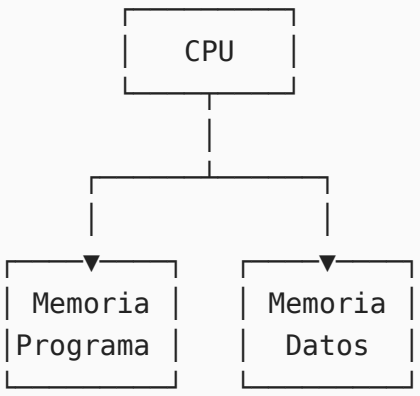


Harvard

Utiliza memorias separadas para almacenar las instrucciones del programa y los datos.

- Memoria de programa independiente.
- Memoria de datos independiente.

- Puede acceder a instrucciones y datos simultáneamente.
- Permite mayor velocidad de acceso en determinadas operaciones.



3. Memorias

Memoria	Característica	Función
Flash	No volátil	Almacena el programa o firmware.
SRAM	Volátil	Guarda variables y datos mientras el programa se ejecuta.
EEPROM	No volátil	Almacena pequeños datos que deben conservarse al apagar el dispositivo.

- **Flash:** almacena el programa.
- **SRAM:** almacena las variables.
- **EEPROM:** conserva datos después del apagado.

Escritura y borrado

Las memorias no volátiles tienen una cantidad limitada de ciclos de escritura y borrado.

- **Flash:** aproximadamente 10 000 ciclos.
- **EEPROM:** aproximadamente 100 000 ciclos.

Estos valores son orientativos y dependen del componente.

En la memoria Flash, la información se conserva mediante carga eléctrica atrapada en estructuras especiales de los transistores.

El proceso de escritura y borrado depende de la tecnología:

- Flash: normalmente se borra por bloques y se programa por páginas.
- EEPROM: permite modificar datos en unidades más pequeñas.

4. Unidades de almacenamiento

- 1 byte = 8 bits.
- 1 KB = 1024 bytes.
- 1 MB = 1024 KB.

5. Periféricos

Son módulos que permiten a la CPU interactuar con el exterior. Trabajan bajo el control de la CPU para recibir información o realizar acciones.

Ejemplos:

- Pines de entrada y salida.
- Sensores.
- Pantallas.
- Temporizadores.
- Interfaces de comunicación.

Una instrucción puede ejecutarse en uno o varios ciclos de reloj. Su duración depende del procesador y de la instrucción.

6. Estructura de una placa ESP32 DevKit V1

La placa está formada por diferentes niveles de integración:

Nivel	Componentes
Placa ESP32 DevKit V1	Conector USB, regulador de voltaje y botones EN y BOOT.
Módulo ESP32-WROOM-32	Chip ESP32, memoria Flash y antena.
Chip ESP32	CPU y recursos internos del microcontrolador.

Componentes importantes:

- **USB:** permite conectar la placa al computador para alimentación y comunicación.
- **Regulador:** adapta el voltaje de entrada para alimentar los componentes.
- **EN:** habilita o reinicia el funcionamiento del ESP32.
- **BOOT:** permite entrar al modo de programación.
- **Antena:** permite la comunicación inalámbrica.

La memoria Flash del módulo puede ser externa al chip ESP32, aunque se encuentra integrada físicamente en el módulo.

7. Bits y niveles de voltaje

Los sistemas digitales representan información mediante dos estados lógicos:

- **LOW (bajo):** representa el bit `0`.
- **HIGH (alto):** representa el bit `1`.

Estado	Arduino Uno	ESP32
LOW (0)	0 V	0 V
HIGH (1)	5 V	3.3 V

El Arduino Uno trabaja con lógica de 5 V, mientras que el ESP32 utiliza lógica de 3.3 V.

Importante: no se deben conectar directamente señales de 5 V a los GPIO del ESP32, ya que no son tolerantes a ese voltaje.

8. Sistema hexadecimal

El sistema hexadecimal utiliza base 16 y permite representar números binarios de manera más compacta.

Sus símbolos son los números del 0 al 9 y las letras de la A a la F.

Decimal	Binario	Hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Nomenclatura: el prefijo `0x` indica que un número está representado en hexadecimal.

Conversión de binario a hexadecimal

Cada grupo de 4 bits representa un dígito hexadecimal.

Ejemplo 1:

```
Binario:       0010 0000
Hexadecimal:      2    0
Decimal:           32
```

Resultado: `00100000 = 0x20 = 32`.

Ejemplo 2:

```
Binario:       1000 0001
Hexadecimal:      8    1
Decimal:          129
```

Resultado: `10000001 = 0x81 = 129`.

9. Compuertas lógicas

Son operaciones que trabajan con valores binarios (0 y 1).

Compuerta	Funcionamiento
AND	Devuelve 1 si ambos valores son 1.
OR	Devuelve 1 si al menos uno es 1.
XOR	Devuelve 1 si los valores son diferentes.
NOT	Invierte el valor: 0 pasa a 1 y 1 pasa a 0.

Tabla de verdad

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

10. Máscaras de bits (Bitmask)

Una máscara es un conjunto de bits que permite seleccionar posiciones específicas dentro de un registro sin modificar innecesariamente los demás bits.

Se utiliza el desplazamiento:

```
1 << N
```

Esto significa desplazar el número binario `1` hacia la izquierda N posiciones.

Ejemplo:

```
N = 0  → 00000001
N = 1  → 00000010
N = 2  → 00000100
N = 3  → 00001000
```

Cada desplazamiento permite seleccionar un bit diferente.

Ejemplo: seleccionar el bit 0.

```
Registro: 00000000
Máscara:  00000001
          --------
Bit:             1
```

La máscara permite trabajar únicamente con el bit seleccionado.

Operaciones con máscaras en C

En los ejemplos se utiliza `REG` como nombre de un registro y `N` como posición del bit.

1. Poner el bit N en 1

```
REG |= (1 << N);
```

- OR con 1 da como resultado 1.
- OR con 0 conserva el valor original.

Activa el bit seleccionado sin modificar los demás.

2. Poner el bit N en 0

```
REG &= ~(1 << N);
```

- `~` invierte los bits de la máscara.

- AND con 0 da como resultado 0.
- AND con 1 conserva el valor original.

Borra el bit seleccionado sin modificar los demás.

3. Invertir el bit N

```
REG ^= (1 << N);
```

- XOR con 1 invierte el bit.
- XOR con 0 conserva el valor original.

Si estaba en 0 pasa a 1, y si estaba en 1 pasa a 0.

4. Preguntar por el bit N

```
if (REG & (1 << N))
```

Realiza una operación AND entre el registro y la máscara.

- Si el resultado es distinto de 0, el bit seleccionado está en 1.
- Si el resultado es 0, el bit seleccionado está en 0.

Ejemplos prácticos

Ejemplo 1: activar el bit 0

```
REG |= (1 << 0);
```

```
REG:      10101010
Máscara:  00000001
          --------
Resultado:10101011
```

El último bit cambia a 1 y los demás permanecen iguales.

Ejemplo 2: borrar el bit 7

```
REG &= ~(1 << 7);
```

```
REG:      10101010
Máscara:  10000000
NOT:      01111111
```

```
           --------
Resultado:00101010
```

El bit 7 pasa a 0 y los demás se conservan.

Ejemplo 3: invertir los cuatro bits inferiores

```
REG ^= 0x0F;
```

Partimos de:

```
REG:      10101010
Máscara:  00001111
           --------
Resultado:10100101
```

Los cuatro bits de la derecha se invierten porque la máscara contiene unos en esas posiciones.

11. Arquitecturas RISC y CISC

Son dos enfoques de diseño de conjuntos de instrucciones de los procesadores.

Característica	RISC	CISC
Instrucciones	Generalmente simples.	Pueden ser más complejas.
Ejecución	Busca instrucciones sencillas y eficientes.	Puede realizar operaciones complejas con menos instrucciones.
Diseño	Conjunto de instrucciones reducido.	Conjunto de instrucciones más amplio.
Ejemplo	AVR.	x86.

No significa que todos los microcontroladores sean RISC y todos los microprocesadores sean CISC. Son características de la arquitectura, no del tipo de dispositivo.

Instrucciones de ensamblador AVR

Instrucción	Función	Ejemplo
LDI	Carga un valor inmediato en un registro.	`LDI R16, 0x20`
ADD	Suma dos registros y guarda el resultado en el primero.	`ADD R16, R17`
SBI	Pone en 1 un bit específico de un registro de E/S.	`SBI 0x05, 5`
RJMP	Realiza un salto relativo a otra instrucción.	`RJMP etiqueta`

Ejemplo: activar un pin de PORTB

En lenguaje C:

```
PORTB |= (1 << 5);
```

En ensamblador AVR:

```
SBI 0x05, 5
```

Ambas instrucciones activan el bit 5 del registro PORTB del ATmega328P.

¿Qué significa `**SBI 0x05, 5**`?

- `SBI` : instrucción para activar un bit.
- `0x05` : dirección de E/S correspondiente a PORTB.
- `5` : posición del bit que se desea activar.

La instrucción se codifica finalmente en lenguaje máquina mediante bits que representan la operación, la dirección del registro y la posición del bit.